IE221 - Kỹ thuật lập trình Python

THƯ VIỆN PANDAS

PHẦN 1: GIỚI THIỆU PANDAS

Pandas là gì?

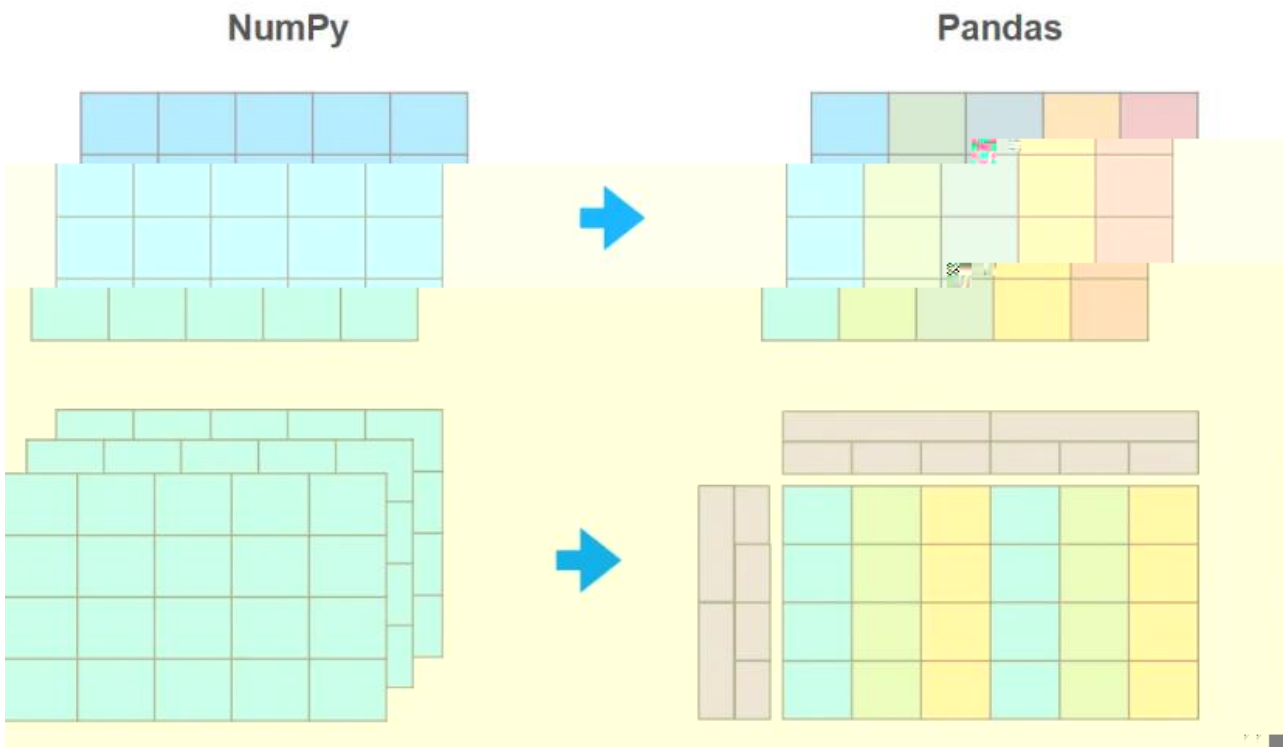
Thư viện phân tích dữ liệu hàng đầu trong Python

Xây dựng trên nền tảng NumPy

Xử lý gigabytes dữ liệu dễ dàng

Công cụ chuẩn cho Data Science và Machine Learning

Tích hợp tốt với visualization libraries



PHẦN 1: GIỚI THIỆU PANDAS

cities.csv:

```
name, population, area
Oslo, 698660, 480.8
Vienna, 1911191, 414.8
Tokyo, 14043239, 2194.1
```

name	population	area
Oslo	698660	480.8
Vienna	1911191	414.8
Tokyo	14043239	2194.1

PHẦN 1: GIỚI THIỆU PANDAS

product id	price, $	weight, g
1	10	100
2	20	150
3	15	200
4	18	80

PHẦN 1: GIỚI THIỆU PANDAS

Sorting

id	price	weight
1	10	100
2	20	150
3	15	200
4	15	80

id	price	weight
1	10	100
3	15	200
4	15	80
2	20	150

numpy array	pandas dataframe
`a[np.argsort(a[:,1])]`	`df.sort_values('price')`

PHẦN 1: GIỚI THIỆU PANDAS

Sorting nhiều cột

id	price	weight
1	10	100
2	20	150
3	15	200
4	15	80

id	price	weight
1	10	100
4	15	80
3	15	200
2	20	150

numpy array

```
b = a[np.argsort(a[:,2])]
b[np.argsort(b[:,1], kind='stable')]
```

pandas dataframe

```
df.sort_values(['price', 'weight'])
```

PHẦN 1: GIỚI THIỆU PANDAS

Thêm cột

id	price	weight
1	10	100
2	20	150
3	15	200
4	15	80

id	price	weight	price per gram
1	10	100	0.1
2	15	80	0.13
3	15	200	0.08
4	20	150	0.19

numpy array	pandas dataframe
`np.column_stack((a, a[:,1]/a[:,2]))`	`df['price_per_gram'] = df.price/df.weight`

PHẦN 1: GIỚI THIỆU PANDAS

Tìm phần tử

df

id	price	weight
100	10	100
314	20	150
213	15	200
432	15	80

df.set_index('id') →

← df1.reset_index()

df1

	price	weight
100	10	100
314	20	150
213	15	200
432	15	80

- read-only
- unique
- pre-heating

numpy array

```
a[np.where(a[0]==213)[0][0]][2]
```

O(N)

pandas dataframe

```
df.loc[df.id==213, 'weight'].item()
```

O(N)

```
df.set_index('id', inplace=True)
df.loc[213, 'weight']
df.reset_index(inplace=True)
```

→ O(1)

PHẦN 1: GIỚI THIỆU PANDAS

Joins by column

sales

client	product_id
John	1
Silvia	3
Andrew	1

products

id	name
1	banana
2	apple
3	orange

client	product_id	name
John	1	banana
Silvia	3	orange
Andrew	1	banana

numpy array	pandas dataframe	sql
_	`sales.join(products.set_index('id'), on='product_id')`	`select * from sales join products on sales.product_id=products.id;`

PHẦN 1: GIỚI THIỆU PANDAS

Grouping by column

sales

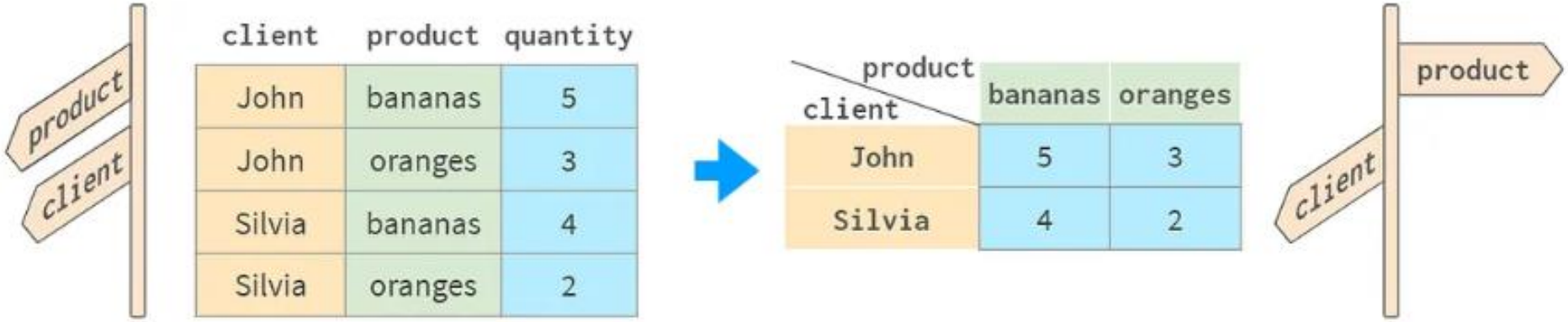
client	product	quantity
John	bananas	5
Silvia	oranges	3
Andrew	bananas	4

product	quantity
bananas	9
oranges	3

numpy array	pandas dataframe	sql
–	`sales.groupby('product', as_index=False)` `.sum('quantity')`	`select product, sum(quantity)` `from sales group by product;`

PHẦN 1: GIỚI THIỆU PANDAS

Pivot tables



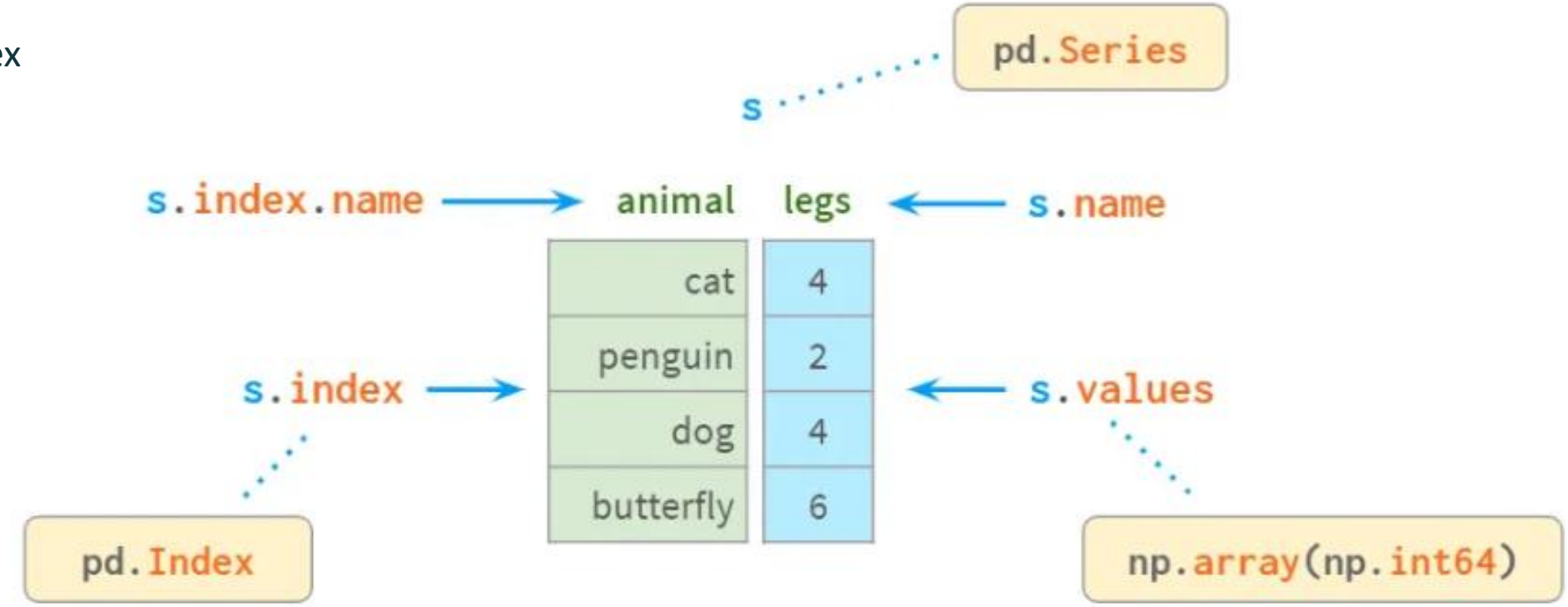
numpy array	pandas dataframe	sql
–	`df.pivot(index='client', columns='product', values='quantity')`	`pivot` (SQL Server, Oracle) `model` clause (Oracle) `crosstab` function (PostgreSQL)

PHẦN 2: SERIES VÀ INDEX

Series - Mảng 1D có Index

Series = NumPy array + Index

Giống như 1 cột trong bảng



PHẦN 2: SERIES VÀ INDEX

```
pd.Series([0.6, 0.7, 2.2, 10.5],
          index=['Athens', 'Oslo', 'Paris', 'Bangkok'])
```

s

Athens	0.6	0
Oslo	0.7	1
Paris	2.2	2
Bangkok	10.5	3

```
s.dtype == np.float64
```

s

Athens	0.6	0
Oslo	0.7	1
Paris	2.2	2
Bangkok	10.5	3

s.Paris
s['Paris']

0.6
0.7
2.2
10.5

s[2:3]

s['Oslo':'Paris']

0.6
0.7
2.2
10.5

s[1:3]

s[:'Oslo']

0.6
0.7
2.2
10.5

s[:2]

PHẦN 2: SERIES VÀ INDEX

.loc[] vs .iloc[]

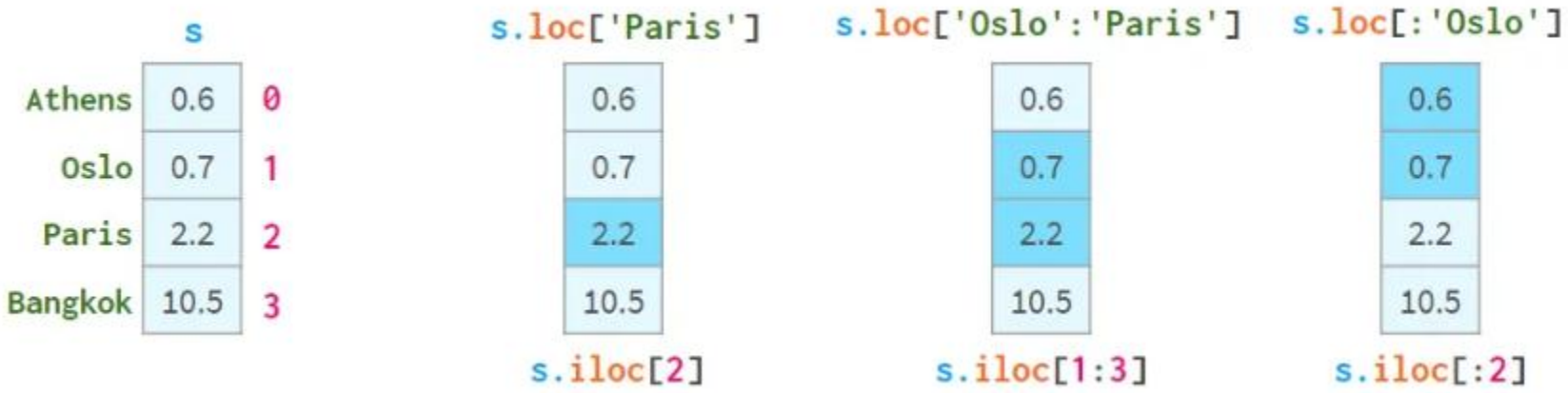
Sự khác biệt chính:

.loc[] - Label-based Indexing

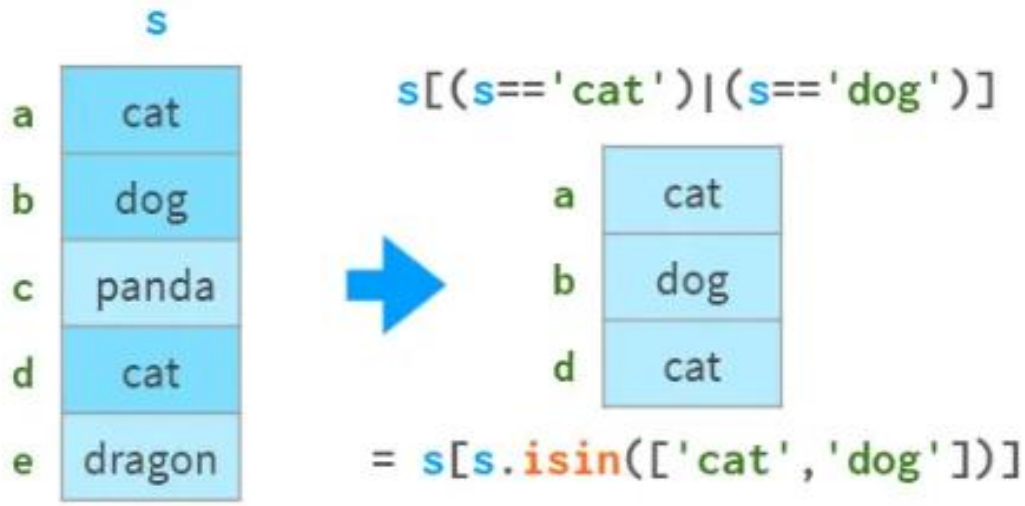
.iloc[] - Position-based Indexing

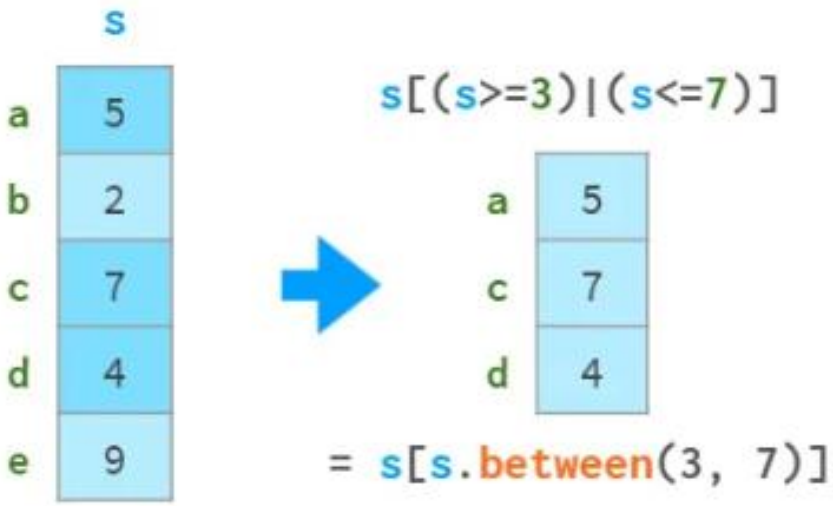
Truy cập theo vị trí số (giống NumPy)

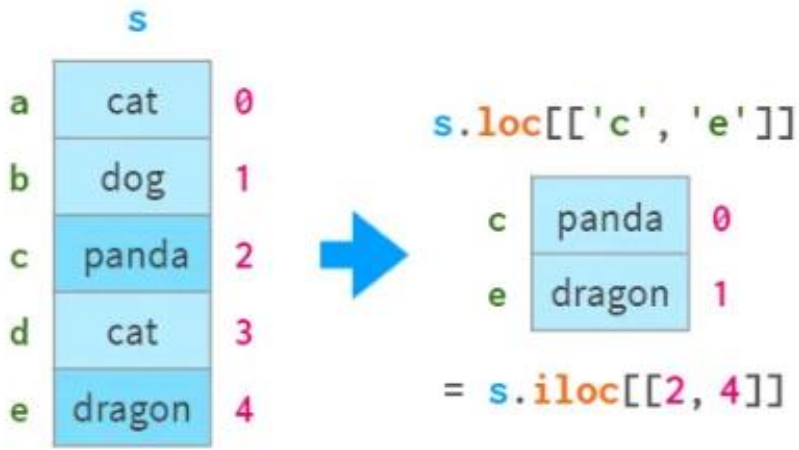
df.iloc[row_position, col_position]



PHẦN 2: SERIES VÀ INDEX







PHẦN 2: SERIES VÀ INDEX

Series vs Pandas

```
s = pd.Series(
    ['cat', 'dog', 'panda', 'cat', 'dragon'],
    index=list('abcde'),
    name='animal'
); s
```

```
a       cat
b       dog
c     panda
d       cat
e    dragon
Name: animal, dtype: object
```

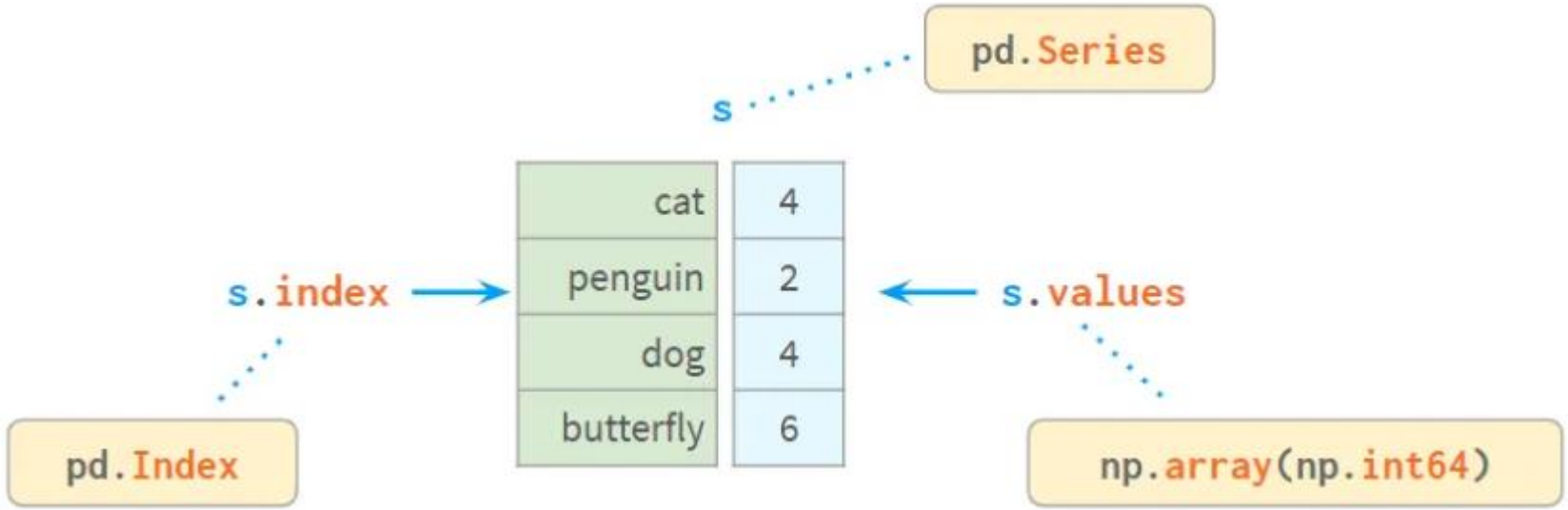
```
s.to_frame()
```

	animal
a	cat
b	dog
c	panda
d	cat
e	dragon

PHẦN 2: SERIES VÀ INDEX

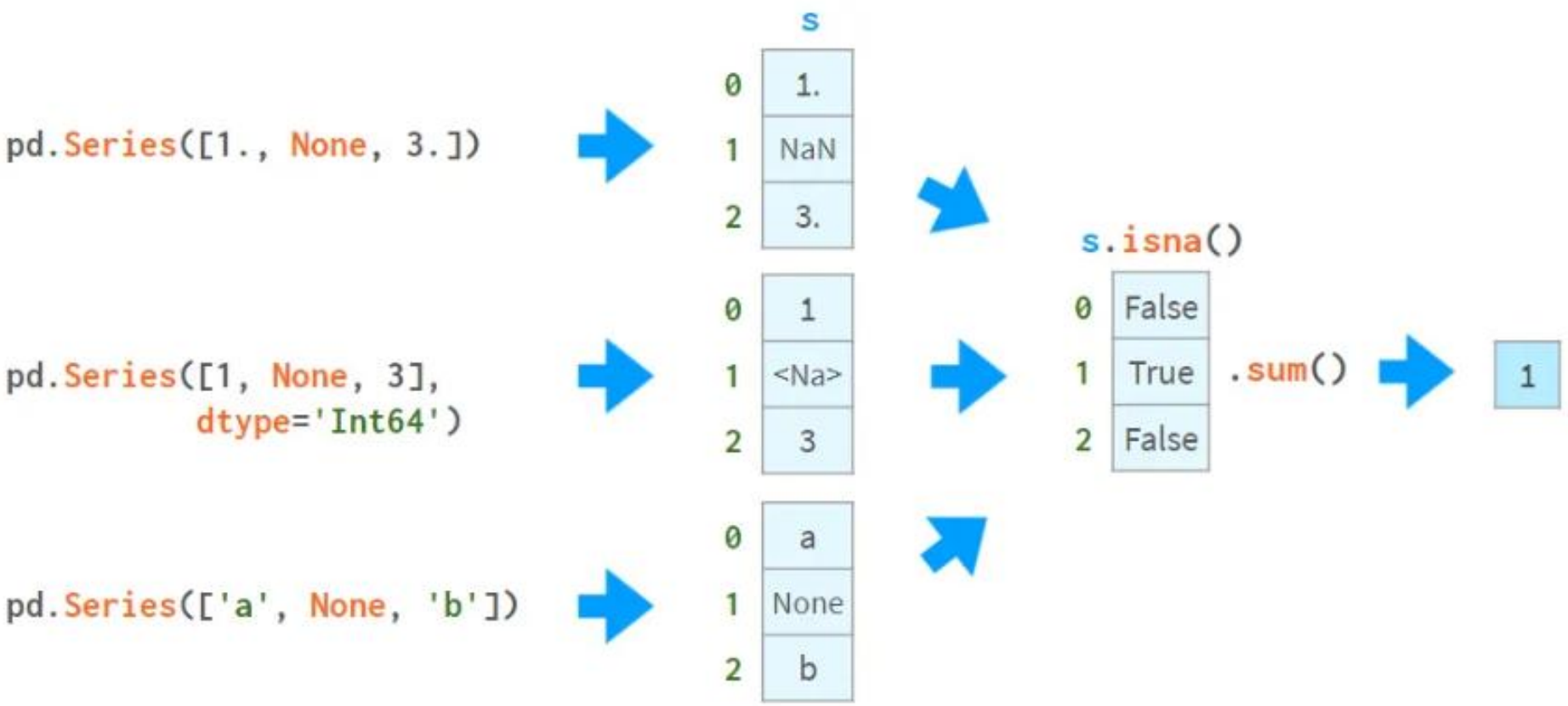
Index – Tìm phần tử bằng value

```
s.index[s.tolist().find(x)]          # faster for len(s) < 1000
s.index[np.where(s.values == x)[0]]  # faster for len(s) > 1000
```



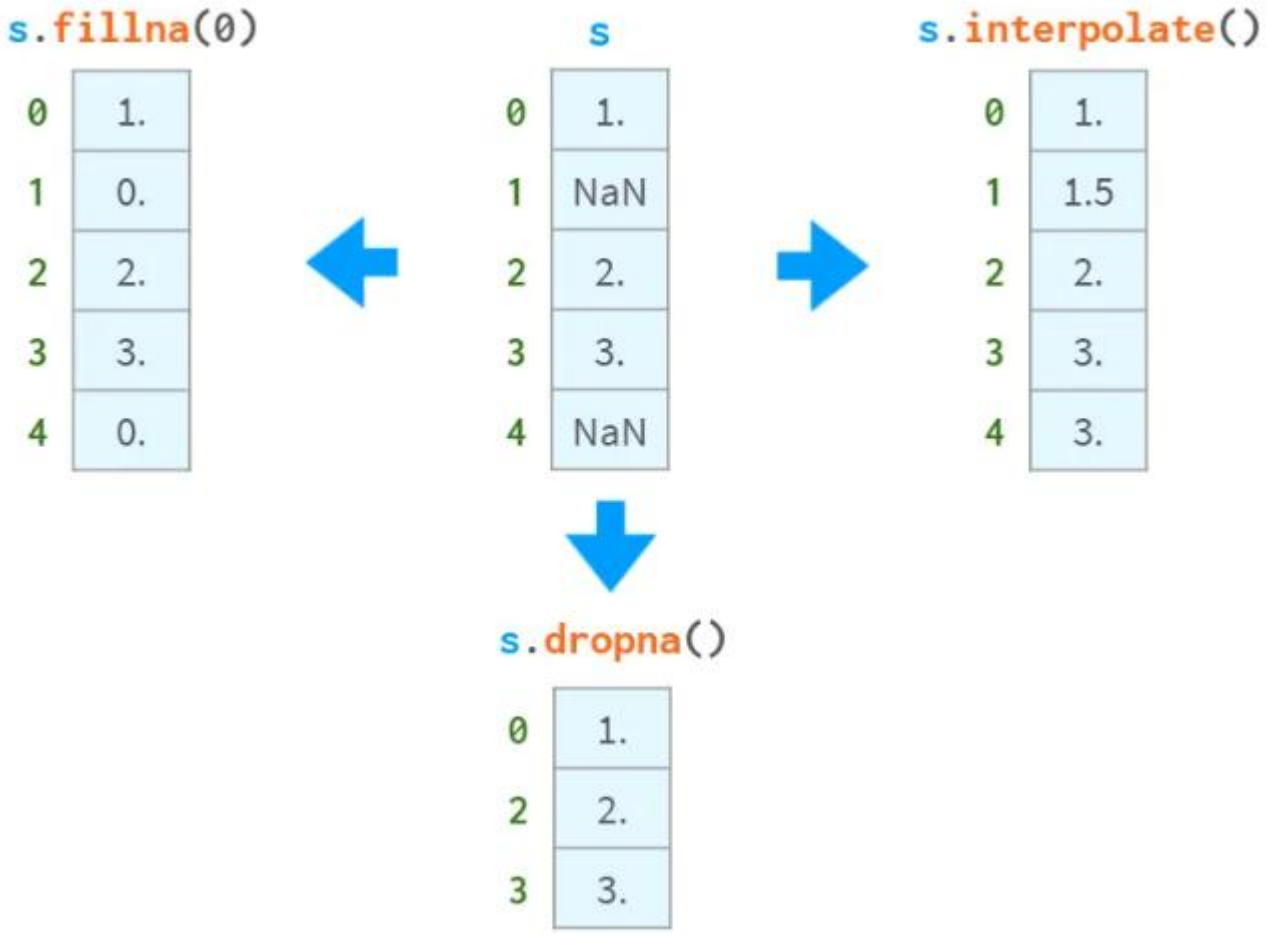
PHẦN 2: SERIES VÀ INDEX

Index – Missing values



PHẦN 2: SERIES VÀ INDEX

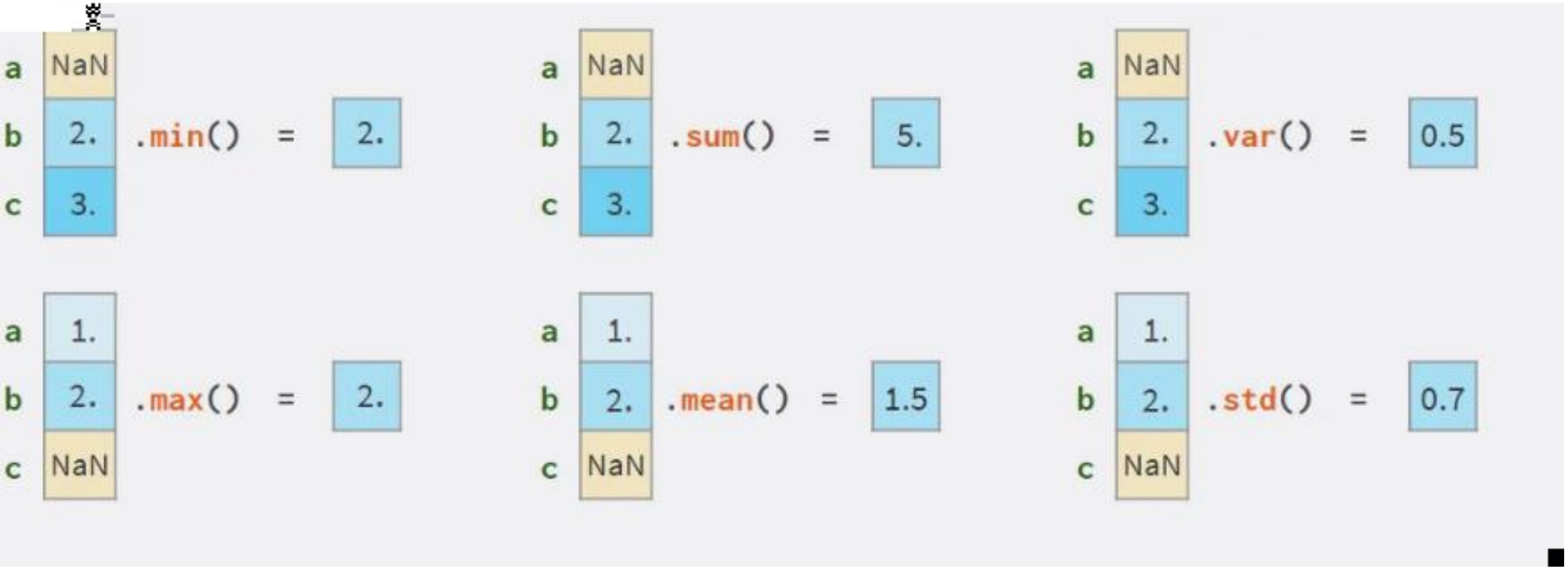
Index – Missing values



fillna(), dropna(), interpolate()

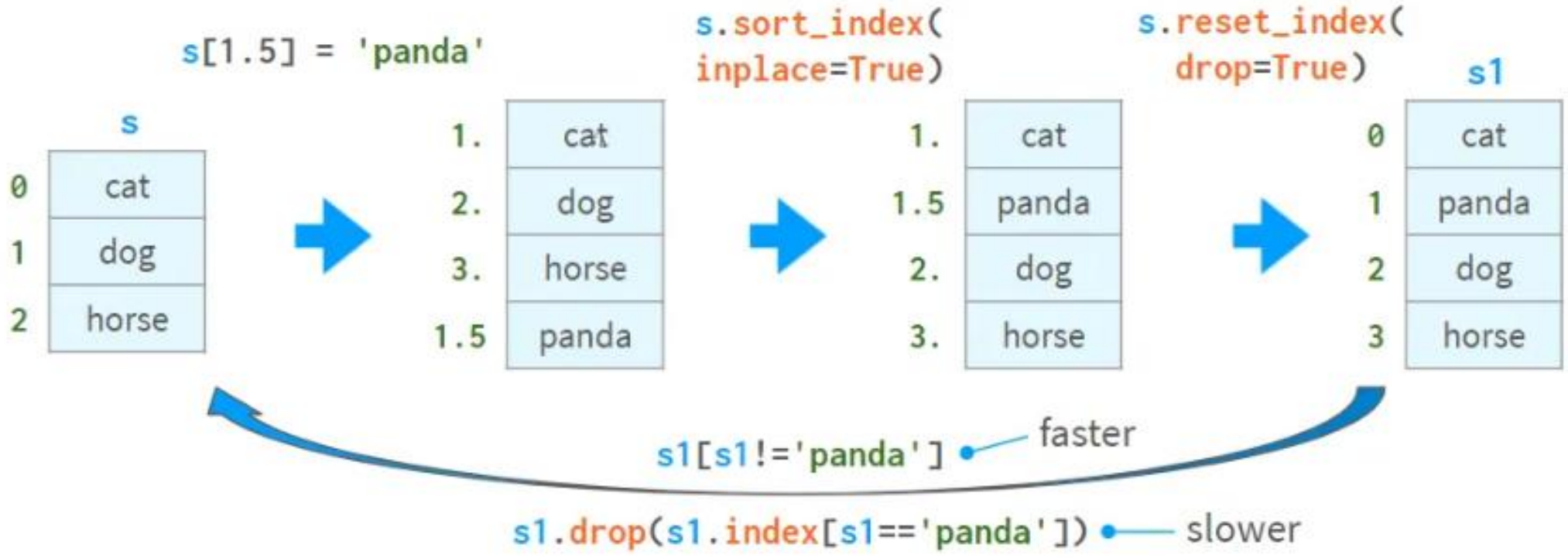
PHẦN 2: SERIES VÀ INDEX

Index – Missing values



PHẦN 2: SERIES VÀ INDEX

Appends, inserts, deletions



PHẦN 2: SERIES VÀ INDEX

Appends, inserts, deletions

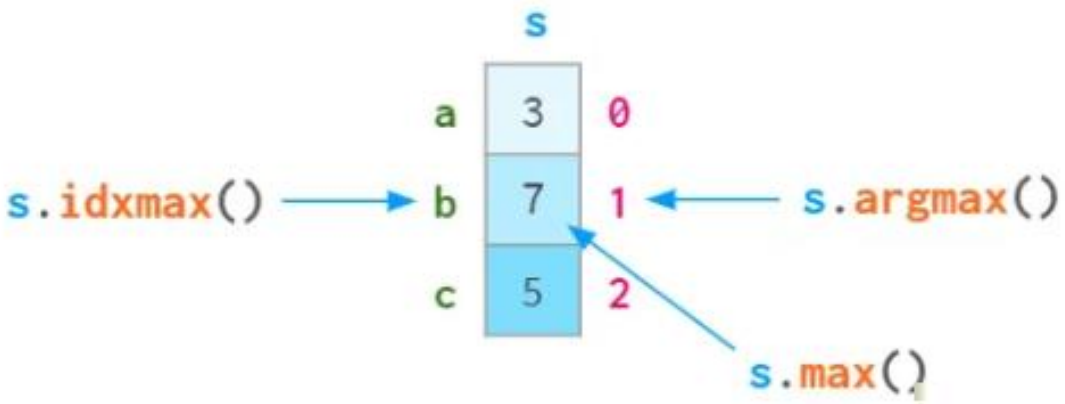
```
pdi.insert(s, 1, 'panda', ignore_index=True)
```

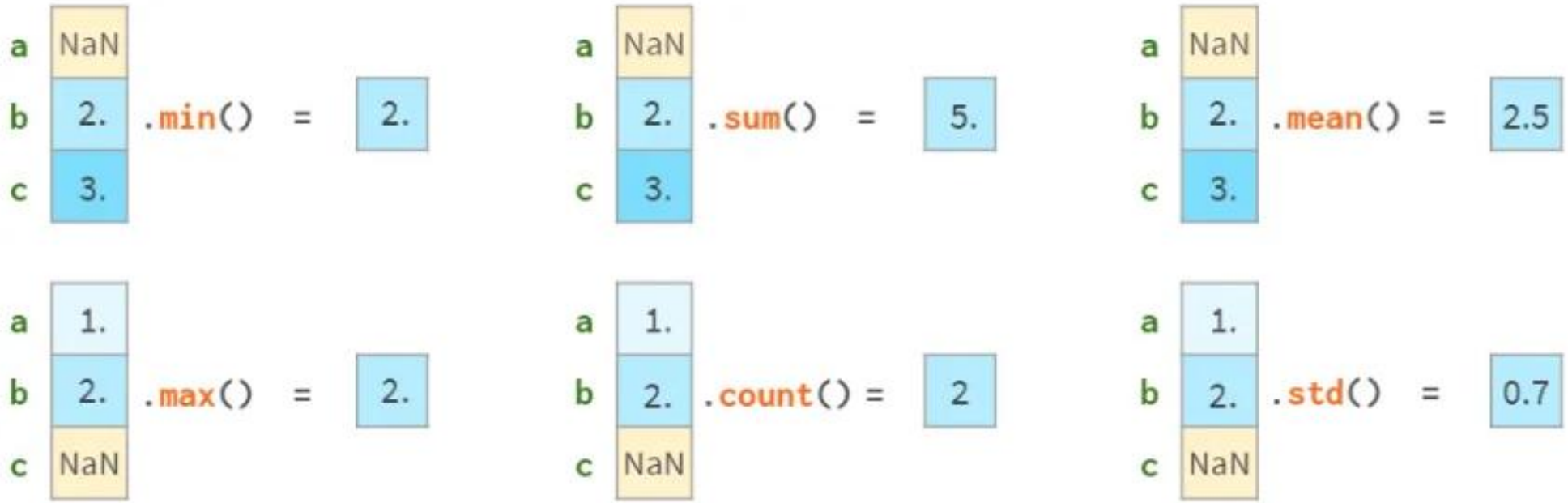
```
pdi.insert(s, 1, 'panda', label=2023)
```

```
i = pdi.find(s, 'dog')
pdi.insert(s, i, 'panda', label=2023)
```

PHẦN 2: SERIES VÀ INDEX

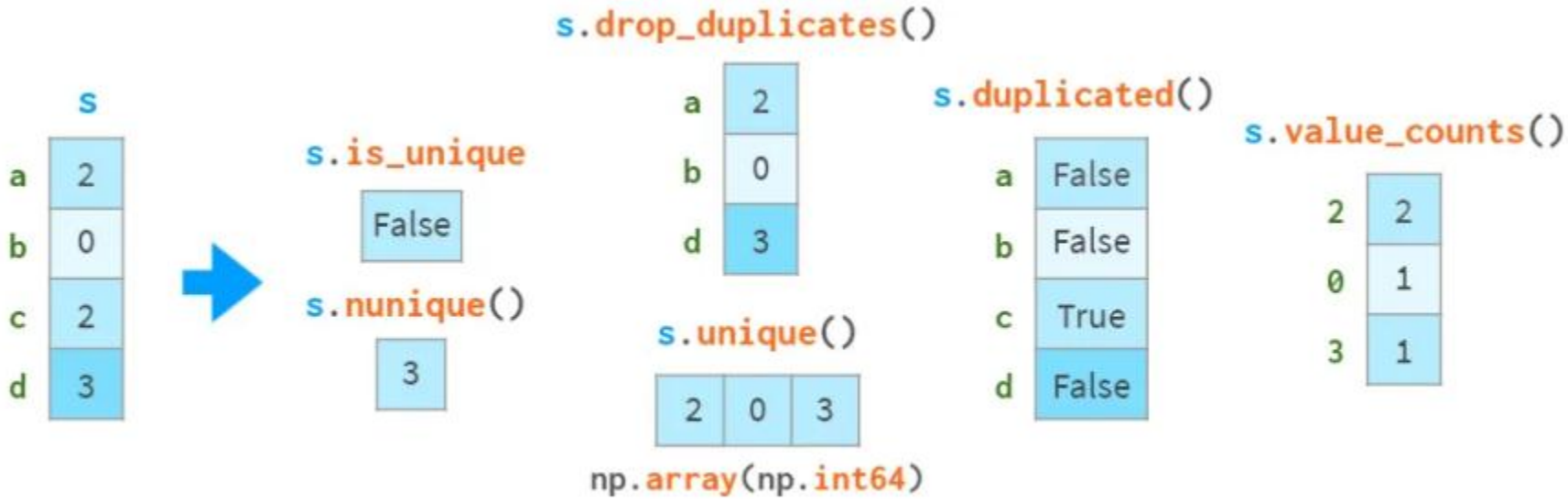
Statistics





PHẦN 2: SERIES VÀ INDEX

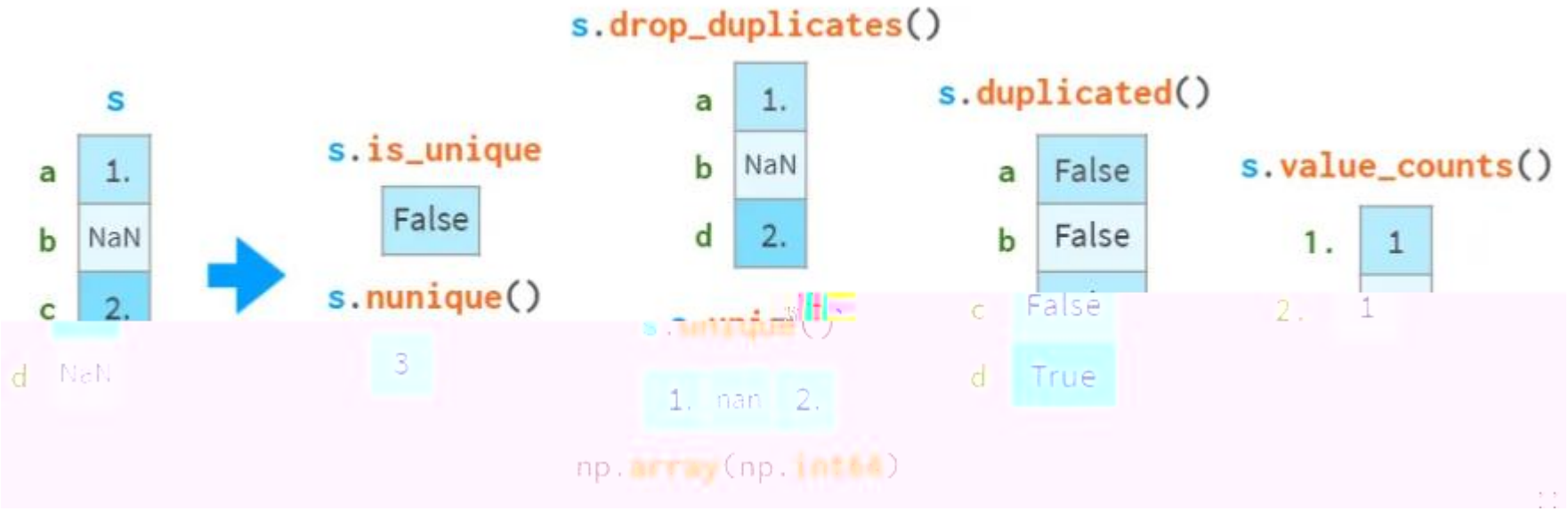
Duplicate data



is_unique, nunique, unique, value_counts

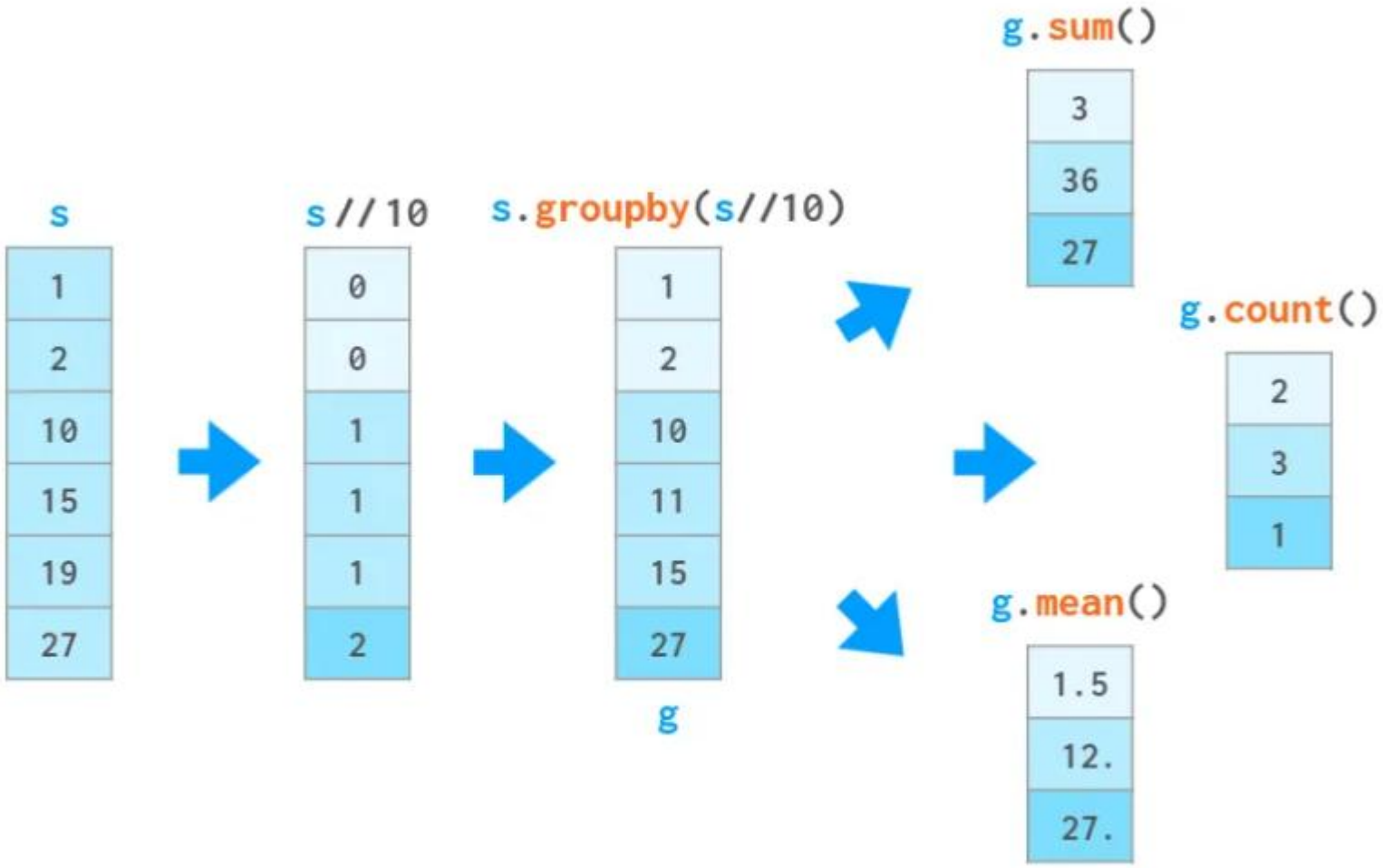
PHẦN 2: SERIES VÀ INDEX

Duplicate data



PHẦN 2: SERIES VÀ INDEX

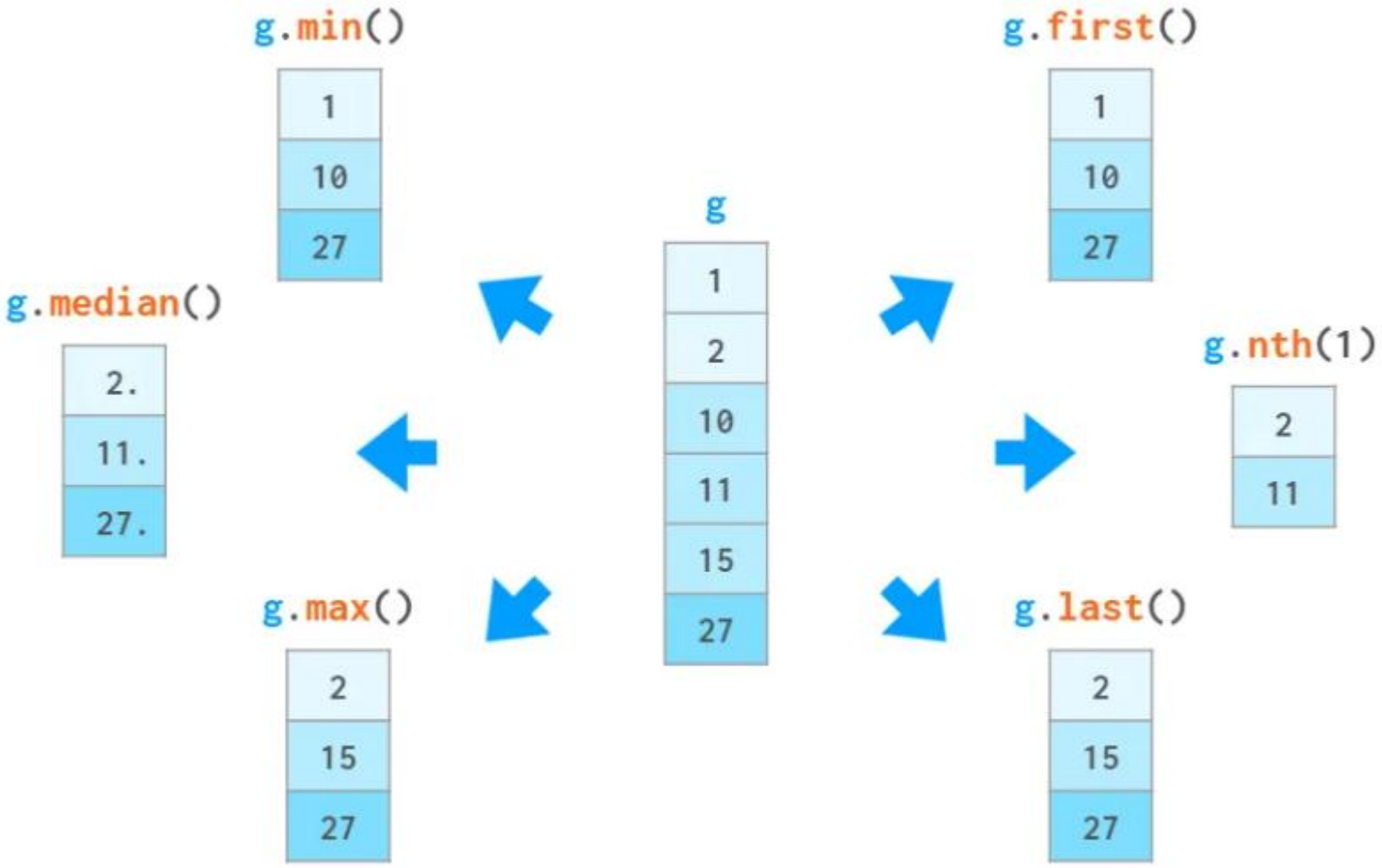
Group by



All operations exclude NaNs

PHẦN 2: SERIES VÀ INDEX

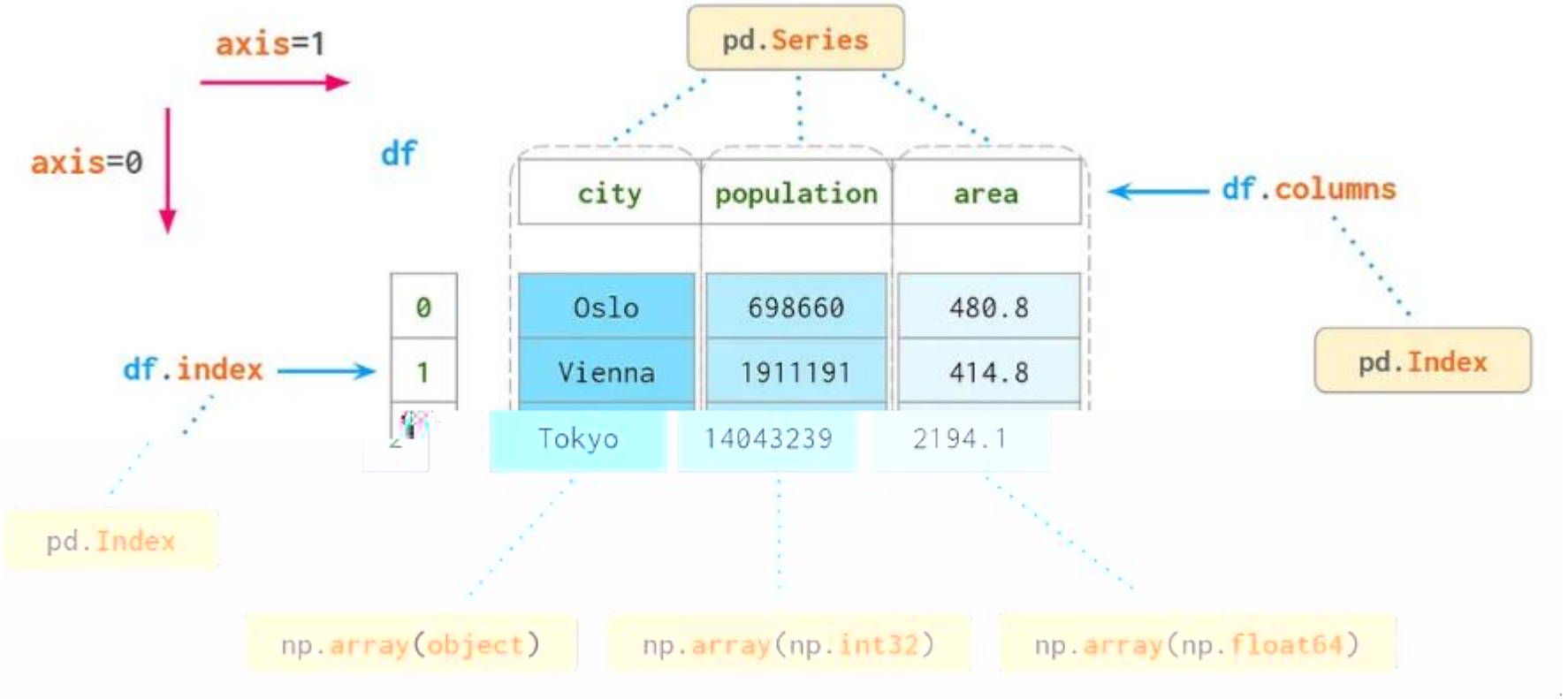
Aggregate functions



min, median, max, first, nth, last

PHẦN 3: DATAFRAMES

Đọc và ghi file csv



PHẦN 3: DATAFRAMES

Đọc và ghi file csv

cities.csv:

csv = comma-separated values

```
name, population, area
Oslo, 698660, 480.8
Vienna, 1911191, 414.8
Tokyo, 14043239, 2194.1
```

header

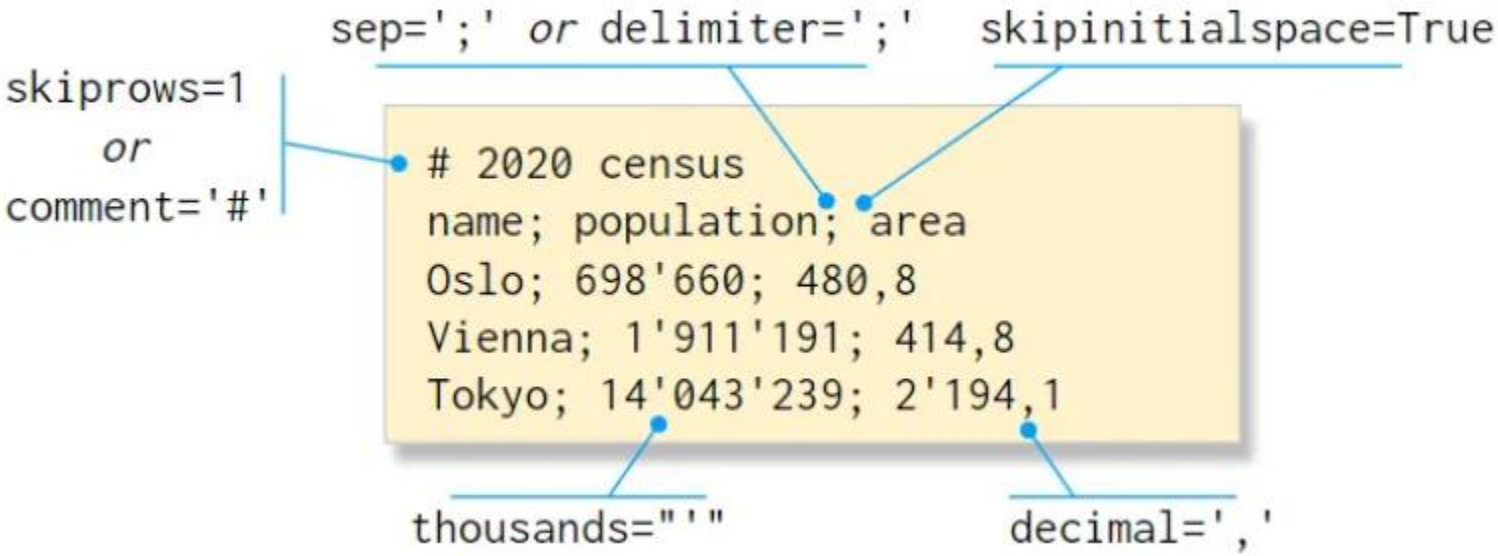
data

```
df = pd.read_csv('cities.csv')
```

	name	population	area
0	Oslo	698660	480.8
1	Vienna	1911191	414.8
2	Tokyo	14043239	2194.1

PHẦN 3: DATAFRAMES

Đọc và ghi file csv



PHẦN 3: DATAFRAMES

Đọc và ghi file csv

```
• pd.read_csv(
      filename,                 # build a dataframe out of this csv-file
      sep=',',                  # or delimiter=',' - field separator
      skipinitialspace=False,   # whether to allow spaces after sep
      quotechar='"',            # used for quoting strings that contain sep
      skiprows=n,               # skip the first n lines (default is 0)
      names=columns,            # use these as column names
      usecols=columns,          # only read these columns
      nrows=n,                  # only read n rows (useful for reading in chunks)
      index_col=columns,        # use column(s) as an index
      ...)                      # ~50 more less frequently used arguments

• df.to_csv(filename, ...)      # write data from the dataframe in a csv format
```

PHẦN 3: DATAFRAMES

Đọc và ghi file csv

```
df = pd.read_csv('cities.csv')
```

	name	population	area
0	Oslo	698660	480.8
1	Vienna	1911191	414.8
2	Tokyo	14043239	2194.1

```
df.set_index('name')
```

```
df = pd.read_csv('cities.csv',
        index_col='name')
```

name	population	area
Oslo	698660	480.8
Vienna	1911191	414.8
Tokyo	14043239	2194.1

PHẦN 3: DATAFRAMES

Tạo Dataframes

dict of lists

```
df = pd.DataFrame({
    'name': ['Oslo', 'Vienna', 'Tokyo'],
    'population': [698660, 1911191, 14043239],
    'area': [480.8, 414.8, 2194.1]
})
```

	name	population	area
0	Oslo	698660	480.8
1	Vienna	1911191	414.8
2	Tokyo	14043239	2194.1

list of lists

```
df = pd.DataFrame([['Oslo', 698660, 480.8],
                   ['Vienna', 1911191, 414.8],
                   ['Tokyo', 14043239, 2194.1]])
```

	0	1	2
0	Oslo	698660	480.8
1	Vienna	1911191	414.8
2	Tokyo	14043239	2194.1

PHẦN 3: DATAFRAMES

Tạo Dataframes

dict of lists

```
df = pd.DataFrame({
      'population': [698660, 1911191, 14043239],
      'area': [480.8, 414.8, 2194.1]
}, index=['Oslo', 'Vienna', 'Tokyo'])
```

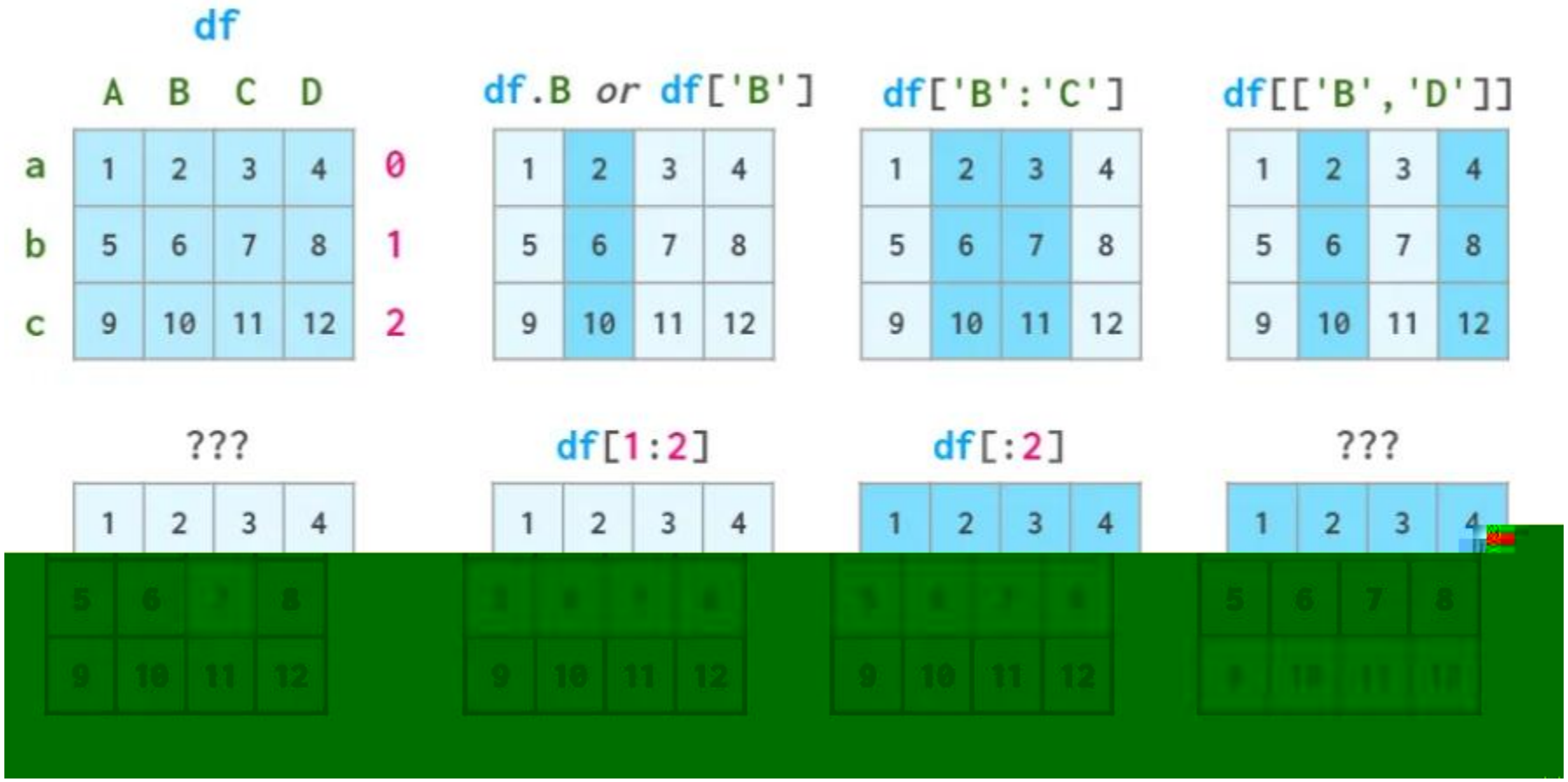
list of lists

```
df = pd.DataFrame(
      [[698660, 480.8],
       [1911191, 414.8],
       [14043239, 2194.1]],
      columns=['population', 'area'],
      index=['Oslo', 'Vienna', 'Tokyo']
)
```

	population	area
Oslo	698660	480.8
Vienna	1911191	414.8
Tokyo	14043239	2194.1

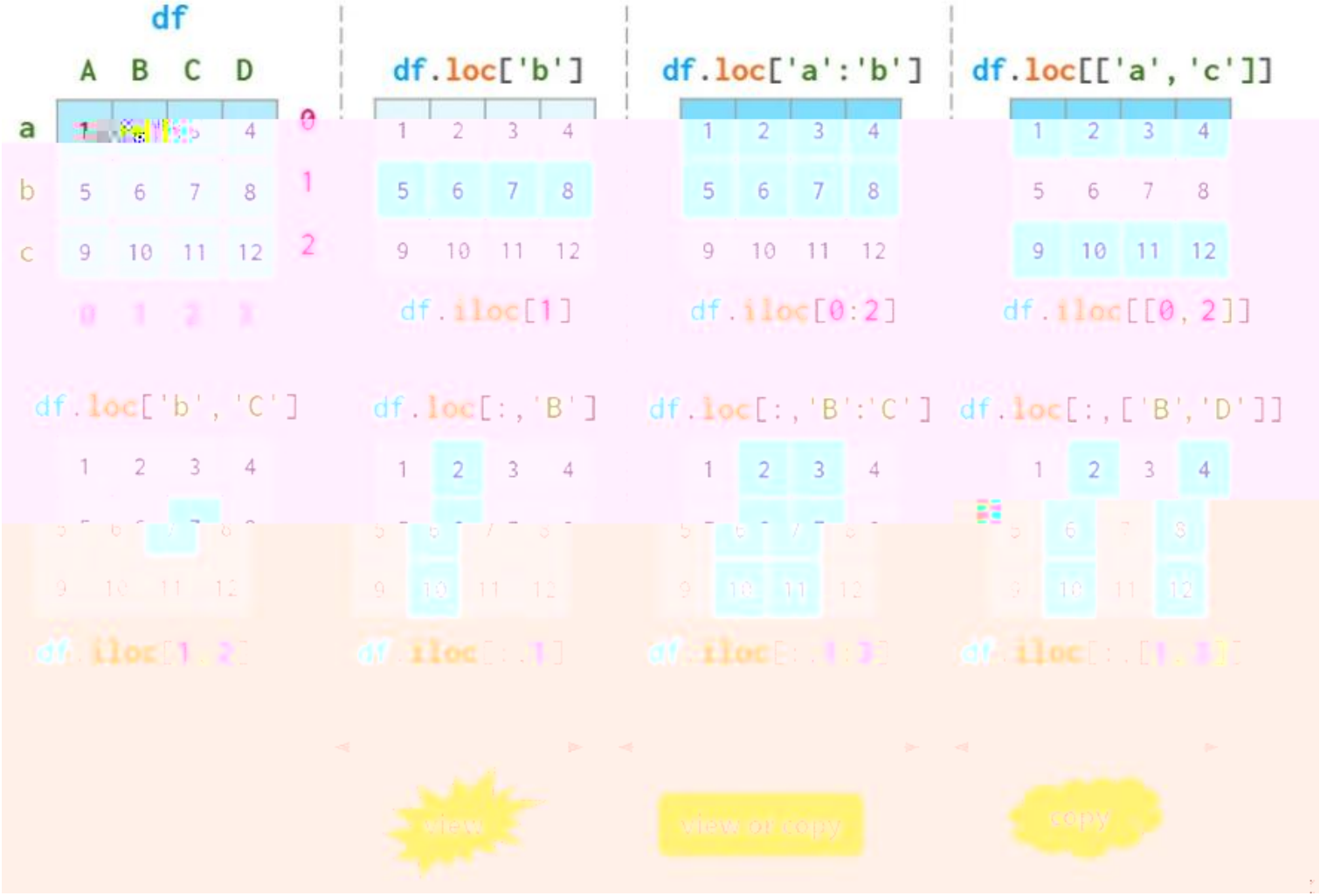
PHẦN 3: DATAFRAMES

Indexing dataframes



PHẦN 3: DATAFRAMES

Indexing dataframes



PHẦN 3: DATAFRAMES

Indexing dataframes

```
df.loc[df.population>10**6, 'name']
```

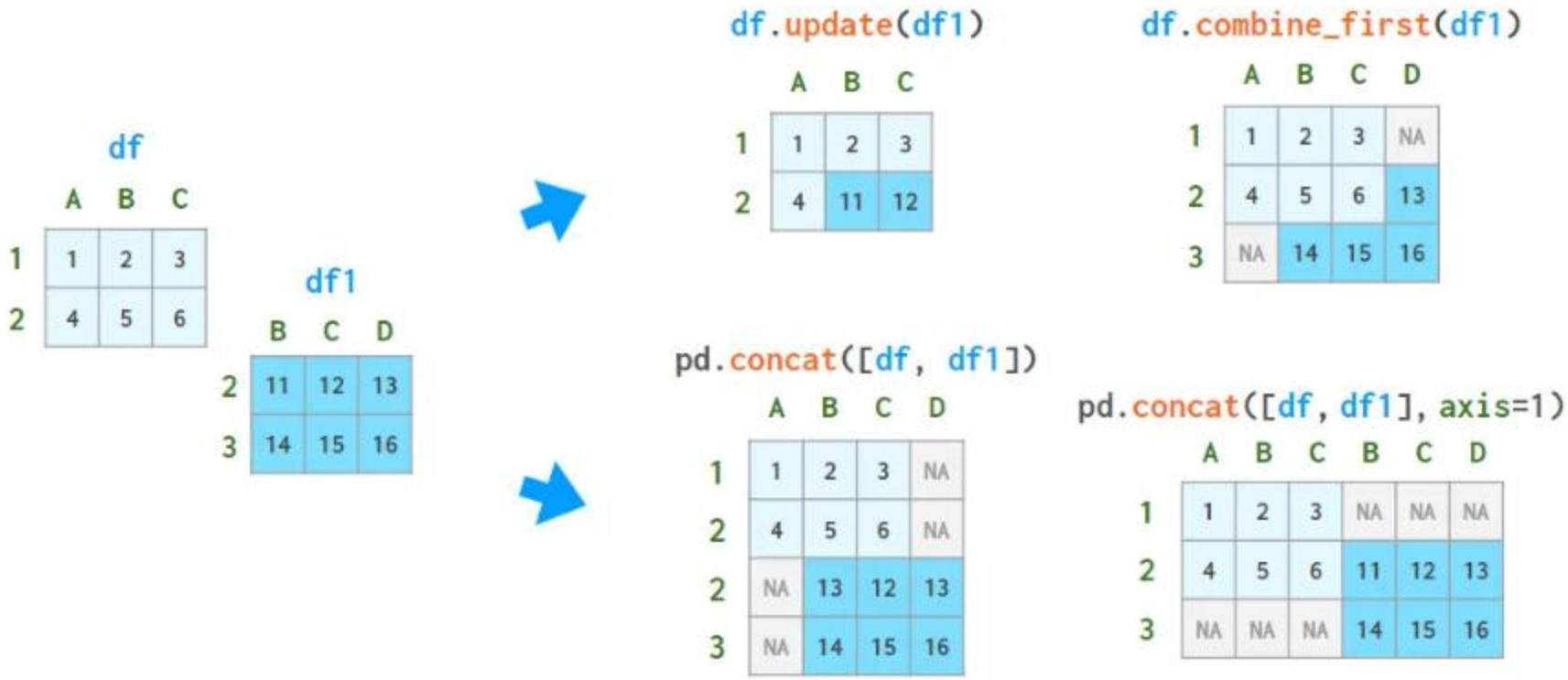
	name	population	area
0	Oslo	698660	480.8
1	Vienna	1911191	414.8
2	Tokyo	14043239	2194.1

	name
1	Vienna
2	Tokyo

```
sql: select name from t where population>1e6;
```

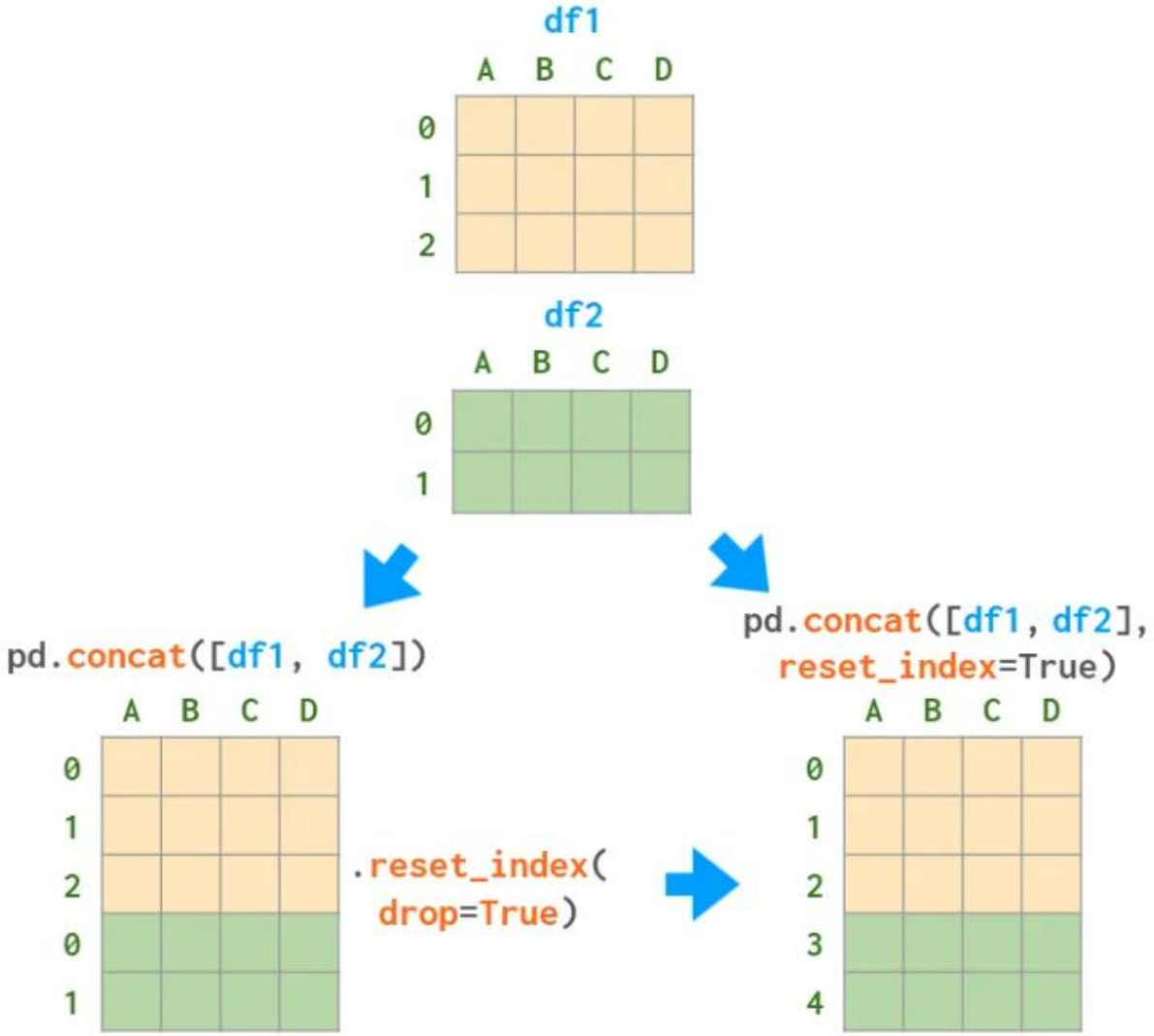
PHẦN 3: DATAFRAMES

Combining dataframes



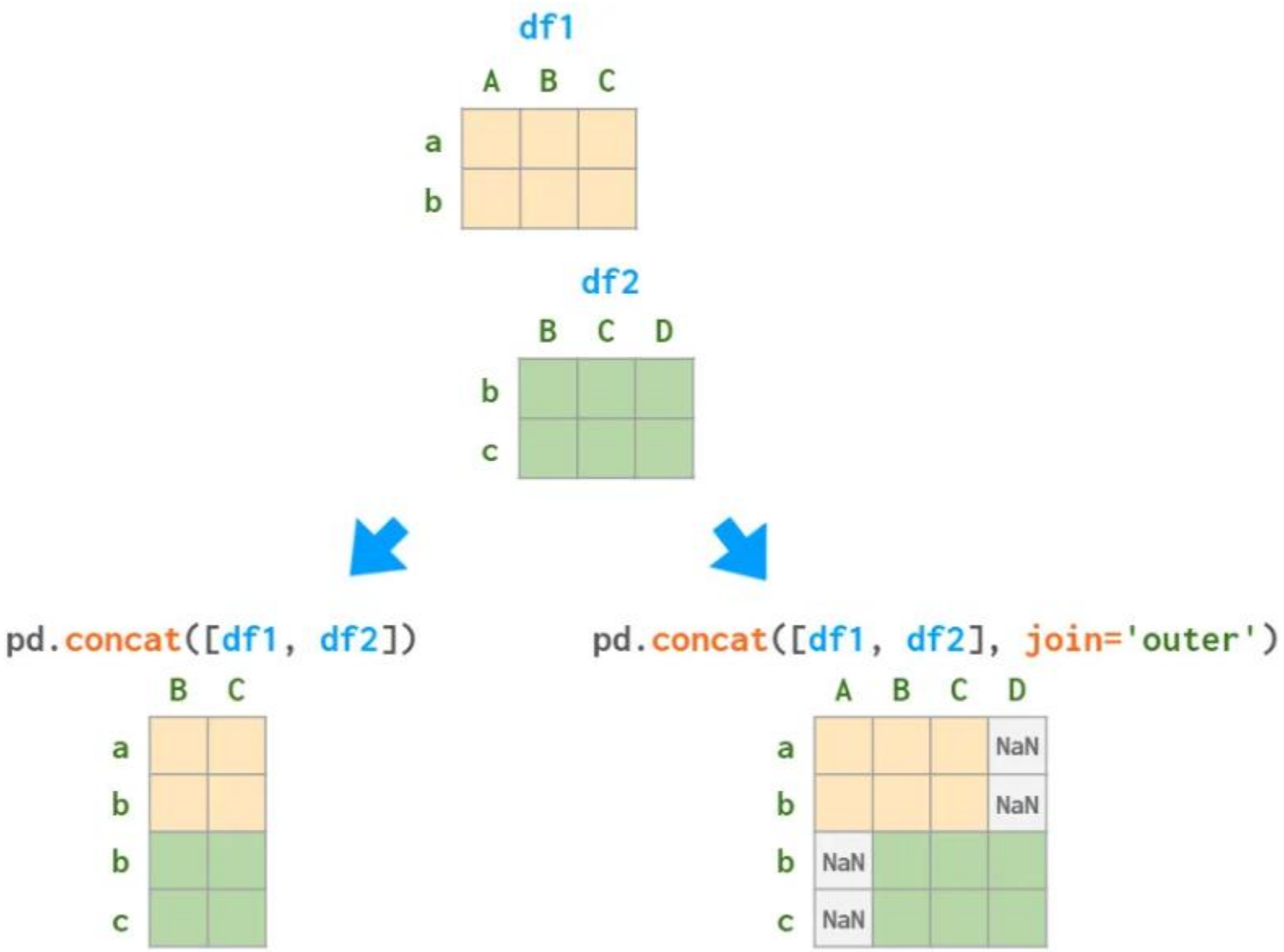
PHẦN 3: DATAFRAMES

Combining dataframes
- Vertical stacking



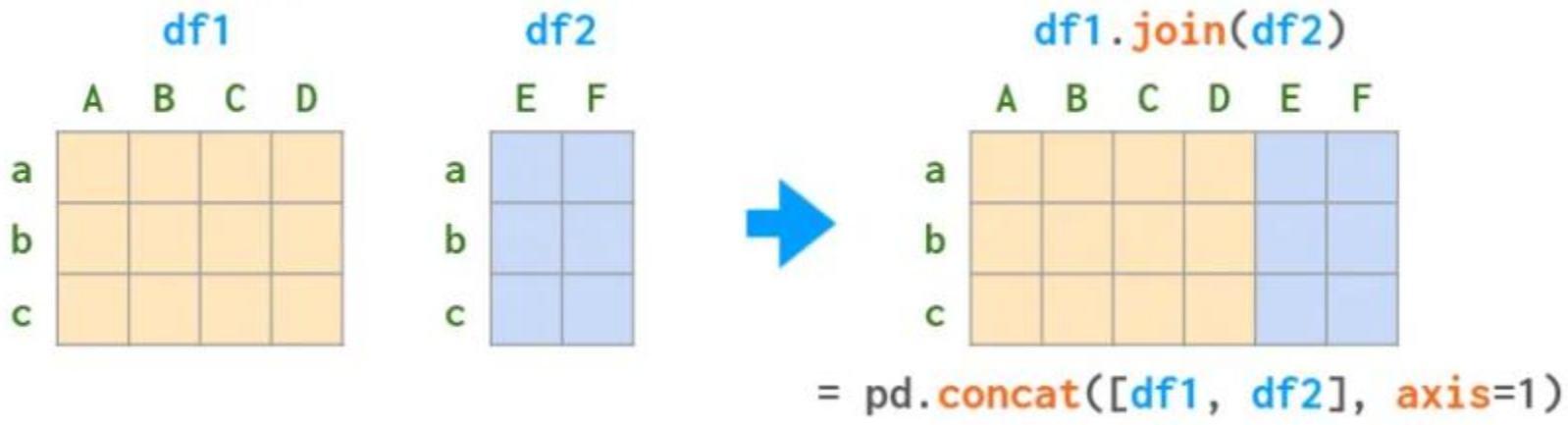
PHẦN 3: DATAFRAMES

Combining dataframes
- Vertical stacking

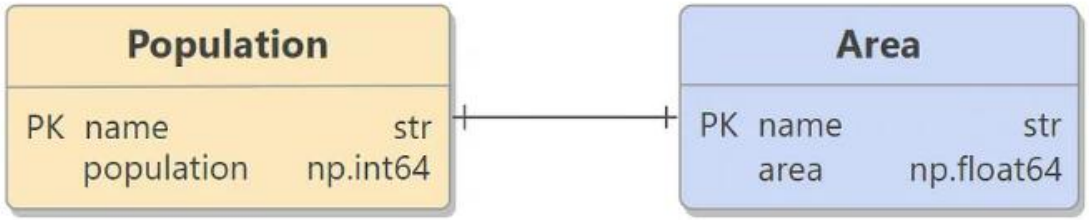


PHẦN 3: DATAFRAMES

Combining dataframes - Horizontal stacking



PHẦN 3: DATAFRAMES



Joining dataframes
- 1:1 relationship joins

df

	name	population
10	Oslo	698660
20	Vienna	1911191

df1

	name	area
30	Vienna	414.8
40	Tokyo	2194.1

```
df.merge(df1, on='name')
```

	name	population	area
0	Vienna	1911191	414.8

inner join

```
..., how='left')
```

	name	population	area
0	Oslo	698660.0	NaN
1	Vienna	1911191	414.8

left outer join

```
..., how='outer')
```

	name	population	area
0	Oslo	698660.0	NaN
1	Vienna	1911191	414.8
2	Tokyo	NaN	2194.1

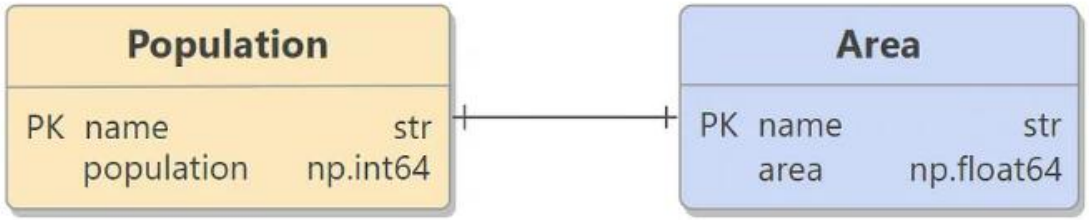
full outer join

```
..., how='right')
```

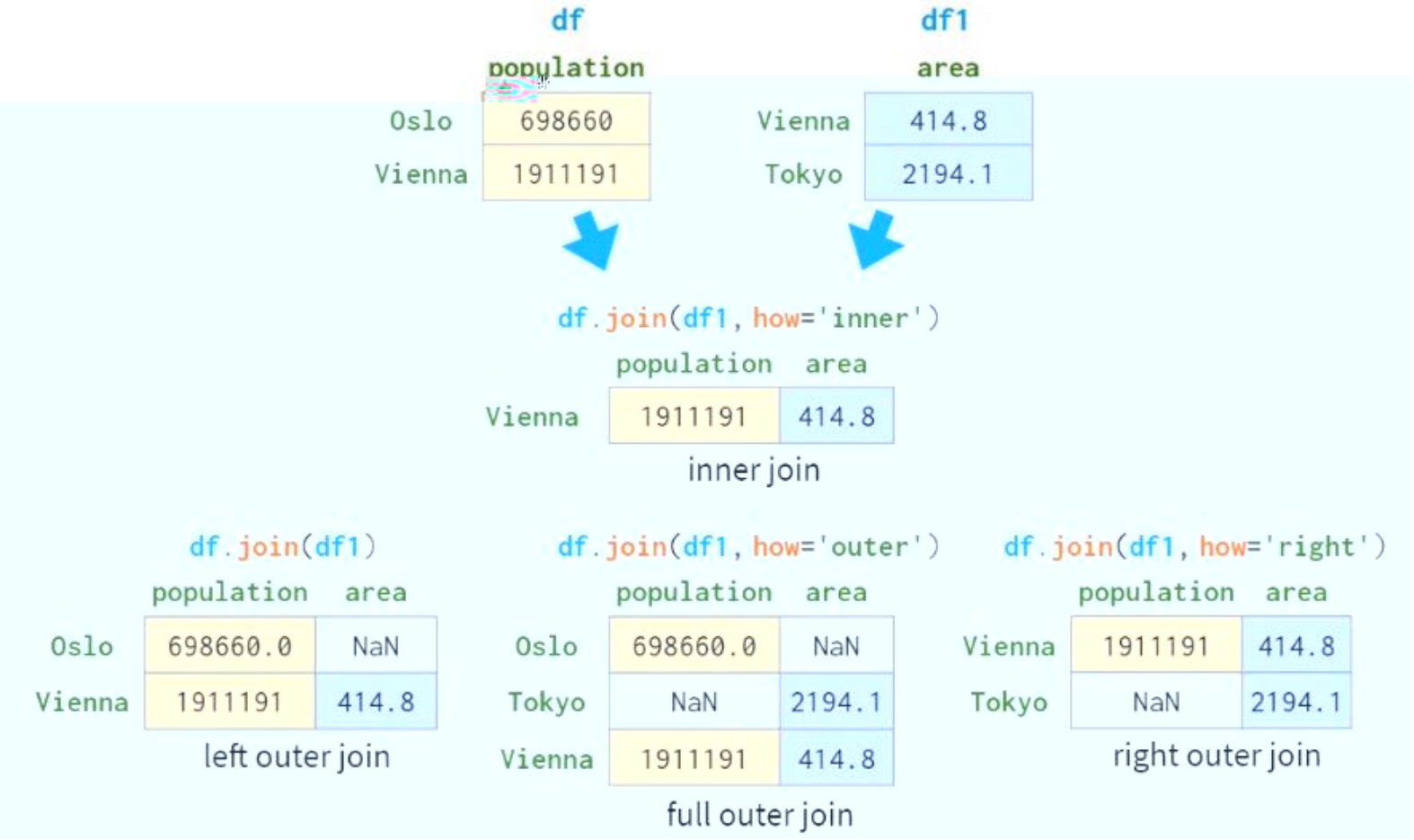
	name	population	area
0	Vienna	1911191	414.8
1	Tokyo	NaN	2194.1

right outer join

PHẦN 3: DATAFRAMES

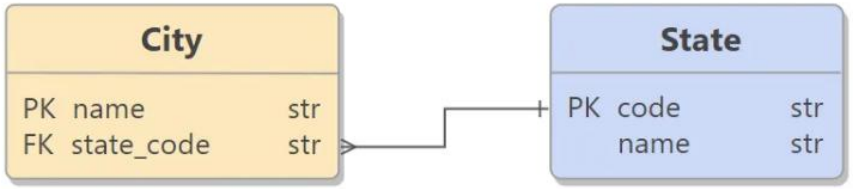


Joining dataframes
- 1:1 relationship joins



PHẦN 3: DATAFRAMES

Joining dataframes
- 1:n relationship joins



cities

	city	state_code
0	San Francisco	CA
1	Miami	FL
2	Washington	DC
3	Los Angeles	CA

states

	code	state
0	CA	California
1	FL	Florida
2	TX	Texas

```
df = cities.merge(states, left_on='state_code', right_on='code')
```

	city	state_code	code	state
0	Los Angeles	CA	CA	Florida
1	San Francisco	CA	CA	California
2	Miami	FL	FL	California

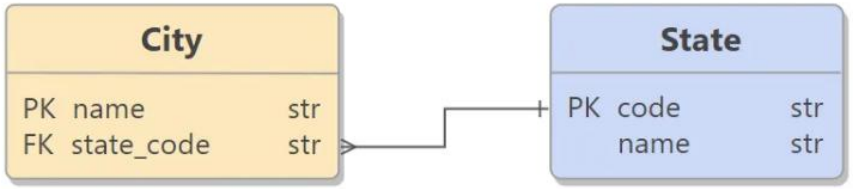
```
df.filter(['city', 'state'])
```

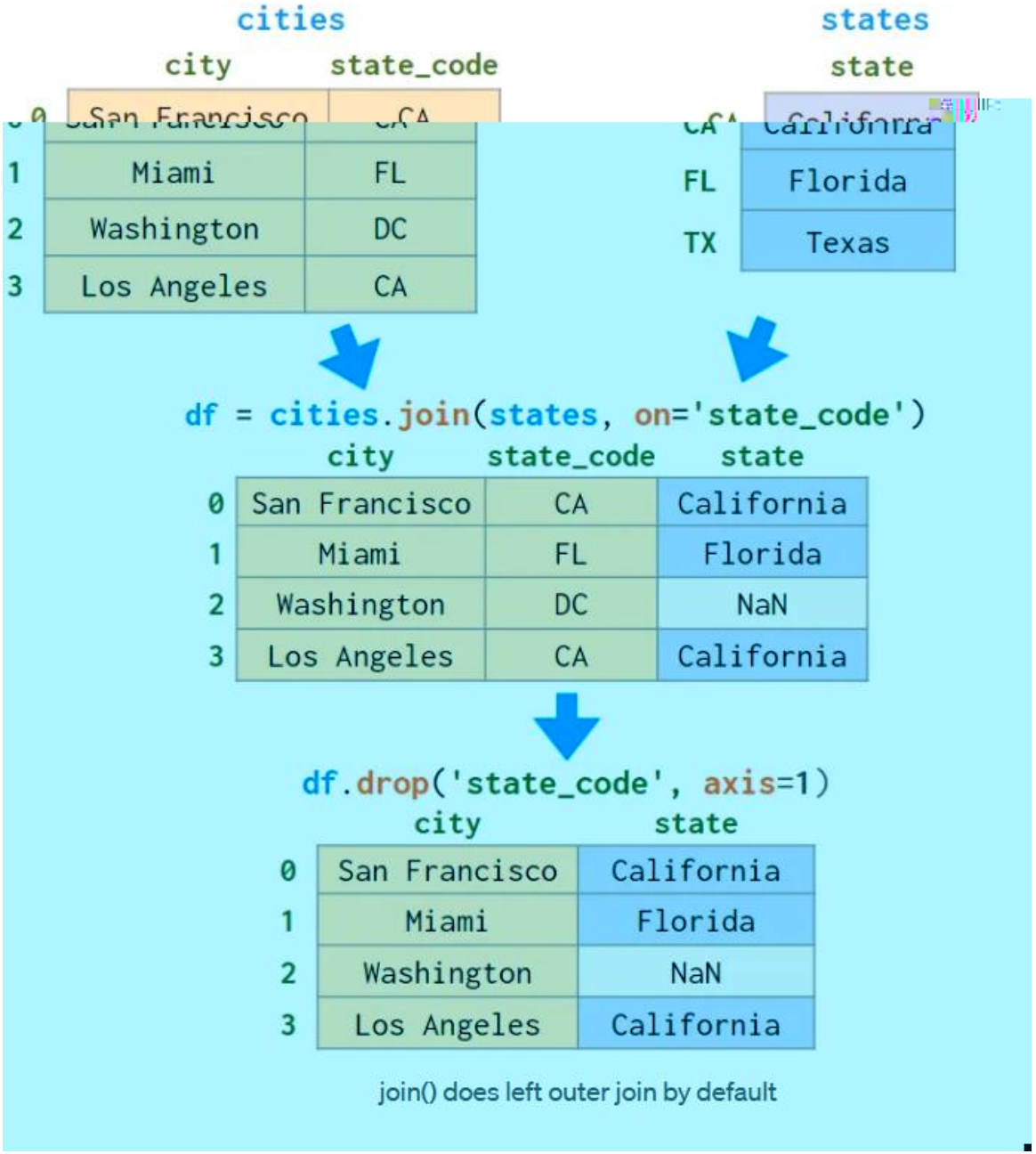
	city	state
0	San Francisco	California
1	Los Angeles	Florida
2	Miami	California

`merge()` performs inner join by default

PHẦN 3: DATAFRAMES

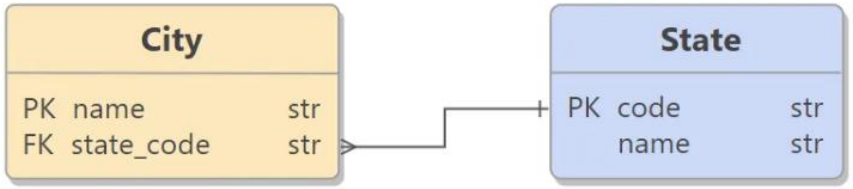
Joining dataframes
- 1:n relationship joins





PHẦN 3: DATAFRAMES

Joining dataframes
- 1:n relationship joins



df

	name	state_code
SF	San Francisco	CA
M	Miami	FL
LA	Los Angeles	CA

df1

	code	name
0	CA	California
1	FL	Florida

```
df.merge(df1, left_on='state_code', right_on='code', suffixes=('_city', '_state'))
```

	name_city	state_code	code	name_state
0	Los Angeles	CA	CA	Florida
1	San Francisco	CA	CA	California
2	Miami	FL	FL	California

```
df.join(df1.set_index('code'), on='state_code', lsuffix='_city', rsuffix='_state')
```

	name_city	state_code	name_state
SF	San Francisco	CA	California
M	Miami	FL	Florida
LA	Los Angeles	CA	California

PHẦN 3: DATAFRAMES

Inserts và deletes

```
df.insert(1, 'D', [7, 8])
```

df

	A	B	C	
a	1	2	3	0
b	4	5	6	1
	0	1	2	

df

	A	D	B	C
a	1	7	2	3
b	4	8	5	6

```
pdi.insert(df, 1, [7, 8, 9], label='c')
```

	A	B	C
a	1	2	3
c	7	8	9
b	4	5	6

PHẦN 3: DATAFRAMES

Inserts và deletes

```
df.drop(columns='D', inplace=True) or
del df['D']
```

df

	A	B	C
a	1	2	3
b	4	5	6

df

	A	D	B	C
a	1	7	2	3
b	4	8	5	6

```
df1.drop(df, index='c', inplace=True)
```

df1

	A	B	C
a	1	2	3
c	7	8	9
b	4	5	6

PHẦN 3: DATAFRAMES

Inserts và deletes

```
pd.concat(df1, df2)
```

df1

	name	population	area
0	Oslo	698660	480.8

df2

	name	population	area
0	Vienna	1911191	414.8
1	Tokyo	14043239	2194.1

df

	name	population	area
0	Oslo	698660	480.8
0	Vienna	1911191	414.8
1	Tokyo	14043239	2194.1

```
df.drop(df[df.name=='Vienna'].index)
```

	name	population	area
2	Tokyo	14043239	2194.1

PHẦN 3: DATAFRAMES

Inserts và deletes

df1

	name	population	area
0	Oslo	698660	480.8

df2

	name	population	area
0	Vienna	1911191	414.8
1	Tokyo	14043239	2194.1

```
pd.concat(df1, df2,
 ignore_index=True)
```

df

	name	population	area
0	Oslo	698660	480.8
1	Vienna	1911191	414.8
2	Tokyo	14043239	2194.1

```
df.drop(df[df.name=='Vienna'].index)
```

	name	population	area
0	Oslo	698660	480.8
2	Tokyo	14043239	2194.1

PHẦN 3: DATAFRAMES

Inserts và deletes